

Manual Técnico - Proyecto Fin de Ciclo DAM



AUTORA: PALOMA ZAPATA VELÁZQUEZ

TUTORA: MERCEDES MARTÍNEZ FRAGOSO

DAM 2025/2026

Índice

1. [Portada](#)
 2. [Introducción](#)
 - 2.5 [Guía para revisión sin código](#)
 3. [Stack tecnológico](#)
 4. [Arquitectura del sistema](#)
 - 4.1 [Visión general](#)
 - 4.2 [Estructura del repositorio](#)
 - 4.3 [Modelo de datos](#)
 - 4.4 [API REST](#)
 - 4.5 [Autenticación y autorización](#)
 - 4.6 [Flujos de negocio \(resumen\)](#)
 - 4.7 [Integración con Stripe \(resumen\)](#)
 - 4.8 [Cliente Flutter \(resumen\)](#)
 - 4.9 [Ciclo de vida de una petición HTTP](#)
 - 4.10 [Flujos detallados con código](#)
 - 4.10.3 [Avatares de perfil](#)
 - 4.11 [Configuración de base de datos](#)
 - 4.12 [Configuración de Stripe \(servidor y móvil\)](#)
 - 4.13 [Buenas prácticas aplicadas](#)
 - 4.14 [Glosario para revisión](#)
 5. [Configuración e instalación](#)
-

1. Portada

Campo	Valor
Nombre del proyecto	SaborSano
Tipo de proyecto	E-commerce académico (productos saludables: alimentos y cosméticos)
Autor(es)	No especificado en el repositorio (<code>package.json</code> del backend declara <code>"author": ""</code>)
Fecha	16 de mayo de 2026

Campo	Valor
Tecnologías principales	Node.js, Express, MySQL, Sequelize, Stripe (test), Flutter/Dart, API REST

Componentes del monorepo:

Componente	Carpeta / identificador	Descripción
Backend API	server-paloma	API REST con Express y ORM Sequelize
Cliente móvil	SaborSano-web (sabor_sano en pubspec.yaml)	Aplicación Flutter orientada a Android/emulador

2. Introducción

2.1 Objetivo del sistema

SaborSano es una plataforma de comercio electrónico que permite a usuarios finales explorar un catálogo de productos saludables, gestionar un carrito de compras, **registrarse e iniciar sesión** con email y contraseña, **subir una foto de perfil (avatar)** en el registro, crear pedidos y consultar el historial de compras, reseñas y estado de envíos. El backend centraliza la lógica de negocio, la persistencia en MySQL y la integración con **Stripe** en modo de prueba para procesar pagos mediante **Payment Intents**.

2.2 Problema que resuelve

Digitaliza la venta de productos saludables eliminando la dependencia de procesos manuales: unifica catálogo, stock, pedidos, envíos y métodos de pago en un único servicio accesible desde una app móvil, con trazabilidad del estado de cada pedido y validación de montos en el servidor antes de cobrar con Stripe.

2.3 Tipo de aplicación

- **Cliente:** aplicación móvil multiplataforma (Flutter), con navegación por rutas nombradas, persistencia local del carrito y sesión de cliente (`ClientSession`), pantallas de login/registro con **foto de perfil opcional**, y cierre de sesión.

- **Servidor:** API REST stateless; autenticación por email/password en registro/login; peticiones posteriores usan header `X-Client-ID` (sin JWT).
- **Datos:** base de datos relacional MySQL gestionada con Sequelize.

2.5 Guía para revisión sin acceso al código

Si no tienes el repositorio abierto, lee en este orden:

1. **Sección 4.9** — ciclo común de toda petición (Flutter → Express → controller → MySQL → respuesta).
2. **Sección 4.10** — casos de uso con diagramas y fragmentos de código (búsqueda, registro, pedido, pago, cron).
3. **Secciones 4.11 y 4.12** — variables y claves de MySQL y Stripe (servidor y móvil).
4. **Sección 4.13** — decisiones de diseño y buenas prácticas.
5. **Sección 5** — cómo levantar el entorno en una máquina de prueba.

Los diagramas usan **Mermaid** (compatible con GitHub, GitLab y muchos visores Markdown).

2.4 Arquitectura general cliente-servidor

```
graph LR
    subgraph Cliente
        A[App Flutter<br/>SaborSano]
        B[SharedPreferences<br/>carrito + sesión]
    end
    subgraph Servidor
        C[Express API<br/>/api/*]
        D[Sequelize ORM]
        E[(MySQL<br/>paloma_db)]
    end
    subgraph Externo
        F[Stripe API<br/>modo test]
    end
    A -->|HTTP JSON| C
    A --- B
    C --> D --> E
    C -->|Payment Intents + Webhooks| F
    F -->|webhook POST| C
```

La app Flutter consume recursos REST bajo el prefijo `/api`. El carrito vive en el dispositivo hasta el checkout; el pedido y sus detalles se persisten en MySQL. El pago con tarjeta usa **Stripe Payment Intents**: el móvil pide un `clientSecret` al servidor y confirma el cobro

con el SDK `flutter_stripe`; el estado final del pedido (`PAGADO` / `PAGO_FALLIDO`) lo actualiza el **webhook** de Stripe en el servidor.

3. Stack tecnológico

3.1 Backend

Tecnología	Versión (referencia en proyecto)	Función
Node.js	Entorno de ejecución	Servidor HTTP y scripts (<code>seed</code> , <code>dev</code>)
Express	<code>^4.18.2</code>	Enrutamiento, middleware, API REST
Sequelize	<code>^6.35.2</code>	ORM y relaciones entre entidades
mysql2	<code>^3.6.5</code>	Driver MySQL para Sequelize
dotenv	<code>^16.3.1</code>	Variables de entorno (<code>.env</code>)
cors	<code>^2.8.5</code>	Habilitar peticiones cross-origin desde la app
morgan	<code>^1.10.0</code>	Logging HTTP en desarrollo
stripe	<code>^22.1.0</code>	SDK oficial: Payment Intents y verificación de webhooks
bcryptjs	<code>^3.0.2</code>	Hash de contraseñas en registro (12 salt rounds)
multer	<code>^2.1.1</code>	Subida de avatar en registro (<code>multipart/form-data</code>)
nodemon	<code>^3.0.2 (dev)</code>	Recarga automática en <code>npm run dev</code>

3.2 Frontend

Tecnología	Versión	Función
Flutter	SDK <code>>=3.0.0</code> <code><4.0.0</code>	UI, navegación y estado de pantallas
Dart	3.x	Lenguaje de la app

Tecnología	Versión	Función
http	^1.2.0	Cliente HTTP hacia la API
shared_preferences	^2.2.2	Persistencia local de perfil de cliente y carrito
webview_flutter	^4.10.0	Pantalla informativa con URL externa
flutter_stripe	^11.5.0	Formulario de tarjeta y confirmación de pago
image_picker	^1.1.2	Selección de foto de perfil en el registro
flutter_lints	^3.0.0 (dev)	Reglas de análisis estático

3.3 Base de datos

Elemento	Detalle
Motor	MySQL
Nombre por defecto	paloma_db (env.example)
Acceso	Host, puerto, usuario y contraseña vía variables DB_*
Sincronización	En NODE_ENV=development, Sequelize ejecuta <code>sequelize.sync({ alter: false })</code> al cargar modelos

3.4 Herramientas y scripts

Comando (server-paloma)	Acción
<code>npm start</code>	Inicia <code>server.js</code> en producción
<code>npm run dev</code>	Inicia con nodemon
<code>npm run seed</code>	Inserta categorías y productos de ejemplo (<code>scripts/seedData.js</code>)
<code>npm run seed:metodos-pago</code>	Inserta métodos de pago de ejemplo (<code>scripts/seedMetodosPago.js</code>)
<code>npm run migrate:cliente-password</code>	Añade columna <code>password</code> a <code>clientes</code> si no existe

Comando (server-paloma)	Acción
<code>npm run migrate:cliente-avatar</code>	Añade columna <code>avatar</code> a <code>clientes</code> si no existe

4. Arquitectura del sistema

4.1 Visión general

El backend sigue una **arquitectura en capas** clásica para APIs Express:

```

flowchart TB
    subgraph Entrada
        R[Routes]
        M1[CORS / Morgan / JSON]
        M2[authMiddleware]
    end
    subgraph Lógica
        C[Controllers]
        S[Services<br/>payment / order]
    end
    subgraph Datos
        MO[Models Sequelize]
        DB[(MySQL)]
    end
    R --> M1 --> M2 --> C
    C --> S
    C --> MO
    S --> MO
    MO --> DB

```

Patrones identificados en el código:

- **Separación de responsabilidades:** rutas → controladores → servicios (pagos/pedidos) → modelos.
- **Manejo centralizado de errores:** `middleware/errorHandler.js` traduce errores de Sequelize y devuelve JSON uniforme `{ success, message }`.
- **Transacciones:** creación de pedidos con `sequelize.transaction()` para atomicidad entre pedido, detalles y descuento de stock.
- **No confiar en el monto del cliente:** `calculateOrderAmountCents` recalcula el total desde la base de datos antes de crear el Payment Intent.

- **Idempotencia Stripe:** cabecera opcional `Idempotency-Key` o clave derivada `order:{orderId}`.
- **Avatares opcionales en registro:** Multer guarda en `public/avatars/`; la BD almacena ruta relativa; Express sirve `/public/*`.

4.2 Estructura del repositorio

```

Paloma/
├── DOCUMENTACION_TECNICA.md
├── server-paloma/
│   ├── server.js                # Punto de entrada, webhook Stripe,
│   └── /api
│       ├── config/database.js   # Instancia Sequelize
│       ├── models/              # Cliente, Producto, Pedido, ...
│       ├── controllers/
│       ├── routes/
│       ├── services/            # payment.service, order.service
│       └── middleware/          # errorHandler, authMiddleware,
uploadAvatar
│   ├── utils/                  # password.js, avatar.js
│   └── public/avatars/         # Imágenes de perfil (servidas en
/public)
│   ├── scripts/                # seedData, migrate password/avatar
│   ├── env.example
│   ├── package.json
│   └── SaborSano-web/
│       ├── lib/
│       │   ├── main.dart        # Rutas de la app
│       │   └── screens/         # login_screen,
register_client_screen, my_orders_screen, ...
│       └── widgets/            # app_bottom_nav (Perfil / Login
dinámico)
│           ├── layouts/
│           ├── theme/
│           └── utils/           # client_validators,
bottom_nav_actions
│   └── services/               # ApiClient, repositorios, sesión,
carrito
│   ├── assets/
│   └── pubspec.yaml

```

4.3 Modelo de datos

Entidades definidas en `server-paloma/models/` y relacionadas en `models/index.js`:

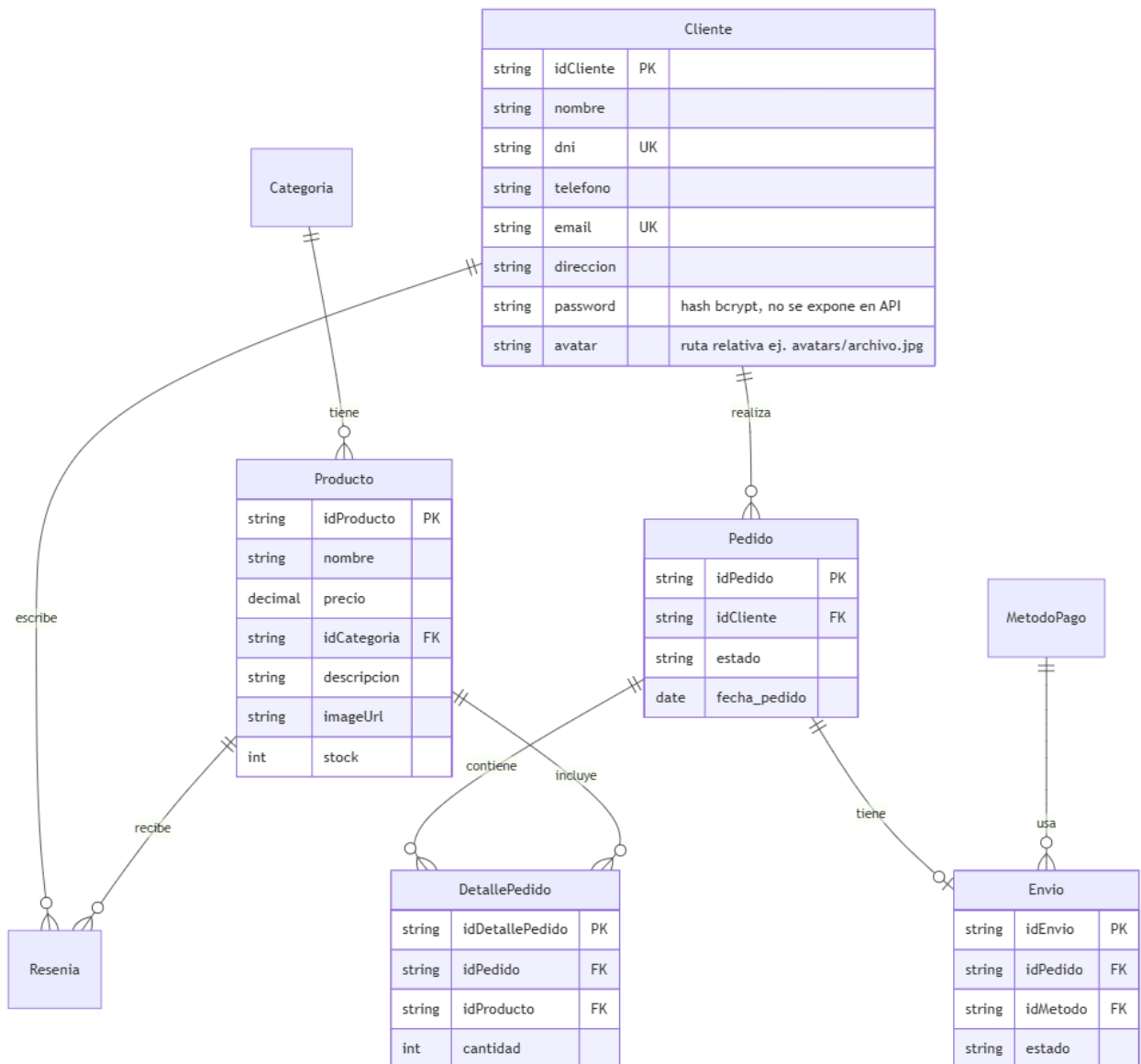


Tabla Sequelize	Nombre físico	Notas
Cliente	clientes	PK CLI...; email único; password bcrypt; avatar ruta relativa bajo /public
Categoria	categoria	
Producto	productos	Incluye stock; se decrementa al crear pedido
Pedido	Pedido	Estados: PENDIENTE, EN PREPARACIÓN, PAGADO, PAGO_FALLIDO
DetallePedido	DetallePedido	
Resenia	resenia	Calificación 1–5, comentario máx. 150 caracteres
Envio	envio	Relación 1:1 con pedido en la práctica

Tabla Sequelize	Nombre físico	Notas
MetodoPago	MetodoPago	Catálogo local (efectivo, tarjetas, etc.)

Los modelos no usan `timestamps` de Sequelize (`timestamps: false`).

4.4 API REST

Base URL: `http://localhost:3000` (puerto por `PORT` o `3000`).

Prefijo API: `/api` (excepto webhook Stripe y archivos estáticos).

Archivos públicos: `/public/*` (avatares en `/public/avatars/`).

Archivos estáticos (avatares)

Elemento	Valor
Carpeta en disco	<code>server-paloma/public/avatars/</code>
URL pública	<code>http://{host}:{PORT}/public/avatars/{nombreArchivo}</code>
Valor en BD (<code>clientes.avatar</code>)	Ruta relativa, ej. <code>avatars/avatar-1710000000-abc.jpg</code>
Subida	Campo multipart <code>avatar</code> en POST <code>/api/clientes</code> (opcional)
Formatos	JPG, PNG, WEBP, GIF — máx. 2 MB
Librería	Multer (<code>middleware/uploadAvatar.js</code>)

La app Flutter **no** recibe la URL completa del API: recibe `avatar` y construye `{ApiClient.baseUrl}/public/{avatar}` con `ApiClient.buildPublicUrl() / ClientProfile.avatarUrl`.

Respuesta estándar

La mayoría de endpoints devuelven:

```
{
  "success": true,
  "data": { },
  "count": 0,
```

```
"message": "texto opcional"
}
```

Endpoints

Método	Ruta	Auth	Descripción
GET	/api	No	Estado de la API (version: 1.0.0)
GET	/api/productos	No	Lista productos; query: idCategoria, q (búsqueda nombre/descripción)
GET	/api/productos/:id	No	Detalle de producto con categoría
GET	/api/categorias	No	Lista categorías
GET	/api/categorias/:id	No	Categoría por ID
POST	/api/clientes	No	Registro: JSON o multipart (nombre, dni, telefono, email, direccion, password, campo opcional avatar)
POST	/api/clientes/login	No	Inicio de sesión con email + password; devuelve perfil (incl. avatar si existe)
GET	/public/avatars/*	No	Sirve la imagen subida (Express static)
GET	/api/clientes/mi-perfil	X-Client-ID	Perfil del cliente autenticado
PUT / PATCH	/api/clientes/mi-perfil	X-Client-ID	Actualizar datos del cliente
POST	/api/pedidos	No	Crear pedido + detalles; body: idCliente, detalles[]
GET	/api/pedidos/mis-pedidos	X-Client-ID	Pedidos del cliente con detalles y envío
GET	/api/pedidos/mis-pedidos/:id	X-Client-ID	Detalle de un pedido propio
GET	/api/resenas	No	Todas las reseñas

Método	Ruta	Auth	Descripción
GET	/api/resenias/ producto/:id	No	Reseñas por producto
GET	/api/resenias/ cliente/:id	No	Reseñas por cliente
GET	/api/resenias/ :id	No	Reseña por ID
POST	/api/resenias	No	Crear reseña
PUT	/api/resenias/ :id	No	Actualizar reseña
DELETE	/api/resenias/ :id	No	Eliminar reseña
GET	/api/metodos- pago	No	Métodos de pago del catálogo
GET	/api/metodos- pago/:id	No	Método por ID
GET	/api/envios	No	Todos los envíos
GET	/api/envios/: id	No	Envío por ID
GET	/api/envios/p edido/:id	No	Envío asociado a un pedido
GET	/api/envios/m is-envios	X-Client -ID	Envíos de pedidos del cliente
POST	/api/payments /create-inten t	X-Client -ID	Crea Stripe Payment Intent; devuelve clientSecret
POST	/api/webhooks /stripe	Firma Stripe	Webhook raw body; eventos de pago

Ejemplo: crear pedido

```

POST /api/pedidos
Content-Type: application/json

{
  "idCliente": "CLI1234567890123",
  "detalles": [
    { "idProducto": "PROD001", "cantidad": 2 }
  ]
}
```

Respuesta 201: pedido en estado PENDIENTE, stock actualizado; mensaje indica envío automático en ~1 minuto.

Ejemplo: Payment Intent

```
POST /api/payments/create-intent
Content-Type: application/json
X-Client-ID: CLI1234567890123

{
  "orderId": "PED1234567890123",
  "currency": "usd",
  "userId": "CLI1234567890123"
}
```

Respuesta:

```
{
  "success": true,
  "paymentIntentId": "pi_XXXXXXXXXXXX",
  "clientSecret": "pi_XXX_secret_XXX"
}
```

El campo amount del body, si se envía, **no se usa** para cobrar; el servidor recalcula centavos con order.service.js.

Ejemplo: registro de cliente

```
POST /api/clientes
Content-Type: application/json

{
  "nombre": "Ana García",
  "dni": "12345678Z",
  "telefono": "612345678",
  "email": "ana@ejemplo.com",
  "direccion": "Calle Mayor 1, Madrid",
  "password": "miClave123"
}
```

Respuesta 201:

```
{
  "success": true,
  "message": "Cliente registrado correctamente",
  "data": {
    "idCliente": "CLI1234567890123",
    "nombre": "Ana García",
    "dni": "12345678Z",
    "telefono": "612345678",
    "email": "ana@ejemplo.com",
    "direccion": "Calle Mayor 1, Madrid",
    "avatar": "avatars/avatar-1710000000-abc123.jpg"
  }
}
```

El campo `password` **no** aparece en la respuesta. El campo `avatar` es opcional (null si no se subió imagen). En servidor la contraseña se almacena como hash bcrypt (`utils/password.js`, 12 salt rounds).

Ejemplo: registro con avatar (multipart)

Alternativa al JSON: `Content-Type: multipart/form-data` con campos de texto y archivo opcional en el campo ``avatar``.

```
POST /api/clientes
Content-Type: multipart/form-data; boundary=---boundary

-----boundary
Content-Disposition: form-data; name="nombre"

Ana García
-----boundary
Content-Disposition: form-data; name="dni"

12345678Z
-----boundary
Content-Disposition: form-data; name="telefono"

612345678
-----boundary
Content-Disposition: form-data; name="email"

ana@ejemplo.com
-----boundary
Content-Disposition: form-data; name="direccion"

Calle Mayor 1, Madrid
-----boundary
```

```
Content-Disposition: form-data; name="password"

miClave123
-----boundary
Content-Disposition: form-data; name="avatar"; filename="foto.jpg"
Content-Type: image/jpeg

(binary)
-----boundary--
```

Errores de subida (tipo no permitido, > 2 MB) → 400 con mensaje descriptivo. Si la validación de negocio falla tras subir el archivo, el servidor elimina la imagen temporal (utils/avatar.js → cleanupUploadedFile).

Ejemplo: inicio de sesión

```
POST /api/clientes/login
Content-Type: application/json

{
  "email": "ana@ejemplo.com",
  "password": "miClave123"
}
```

Respuesta 200 (misma estructura que el registro):

```
{
  "success": true,
  "message": "Sesión iniciada correctamente",
  "data": {
    "idCliente": "CLI1234567890123",
    "nombre": "Ana García",
    "email": "ana@ejemplo.com",
    "dni": "12345678Z",
    "telefono": "612345678",
    "direccion": "Calle Mayor 1, Madrid",
    "avatar": "avatars/avatar-1710000000-abc123.jpg"
  }
}
```

Credenciales incorrectas → 401 con mensaje genérico *"Email o contraseña incorrectos"* (no revela si el email existe).

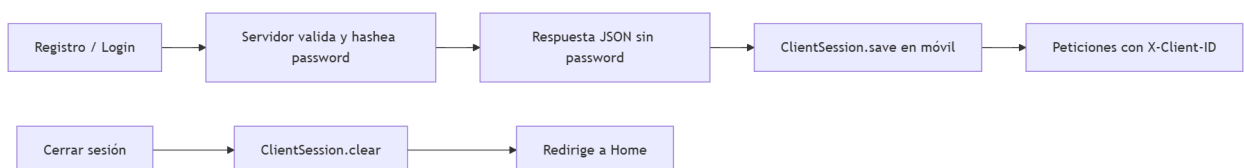
4.5 Autenticación y autorización

No hay JWT ni OAuth en el proyecto. El modelo combina **credenciales (email + password)** en el servidor con **sesión local** en el móvil:

Fase	Mecanismo
Registro	POST /api/clientes con password (JSON o multipart); avatar opcional vía Multer; servidor hashea con bcrypt
Login	POST /api/clientes/login; verifica email (normalizado a minúsculas) y <code>bcrypt.compare</code>
Sesión en app	Tras registro o login, Flutter guarda el perfil en <code>SharedPreferences (ClientSession)</code>
Peticiones protegidas	Header 'X-Client-ID' con <code>idCliente</code> existente en BD (sin token Bearer)
Cierre de sesión	Solo en el móvil: <code>ClientSession.clear()</code> ; no hay endpoint de logout en el servidor

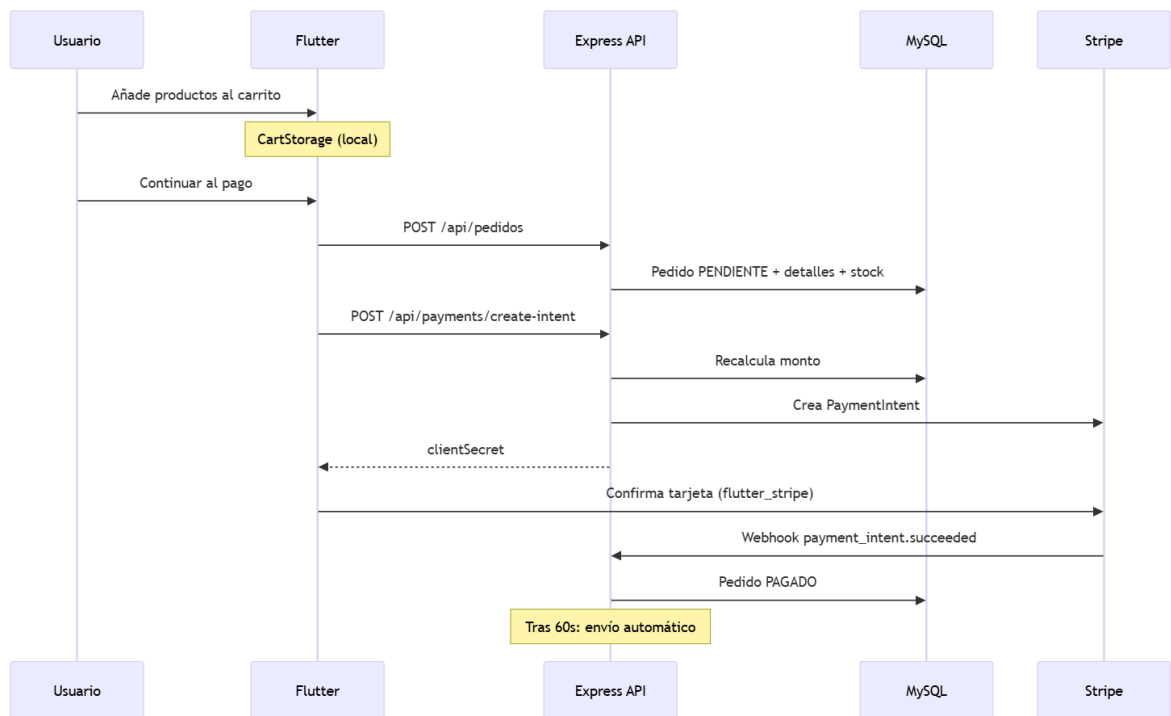
El modelo `Cliente` usa `defaultScope` para **excluir** `password` de todas las consultas normales; el login usa el scope `withPassword` solo para verificar credenciales.

El middleware `authenticate (middleware/authMiddleware.js)` valida presencia, formato no vacío y existencia en BD; expone `req.cliente` y `req.clientId`. Existe además `authorize` para comparar `:id` de ruta con el cliente autenticado (uso limitado en rutas actuales).

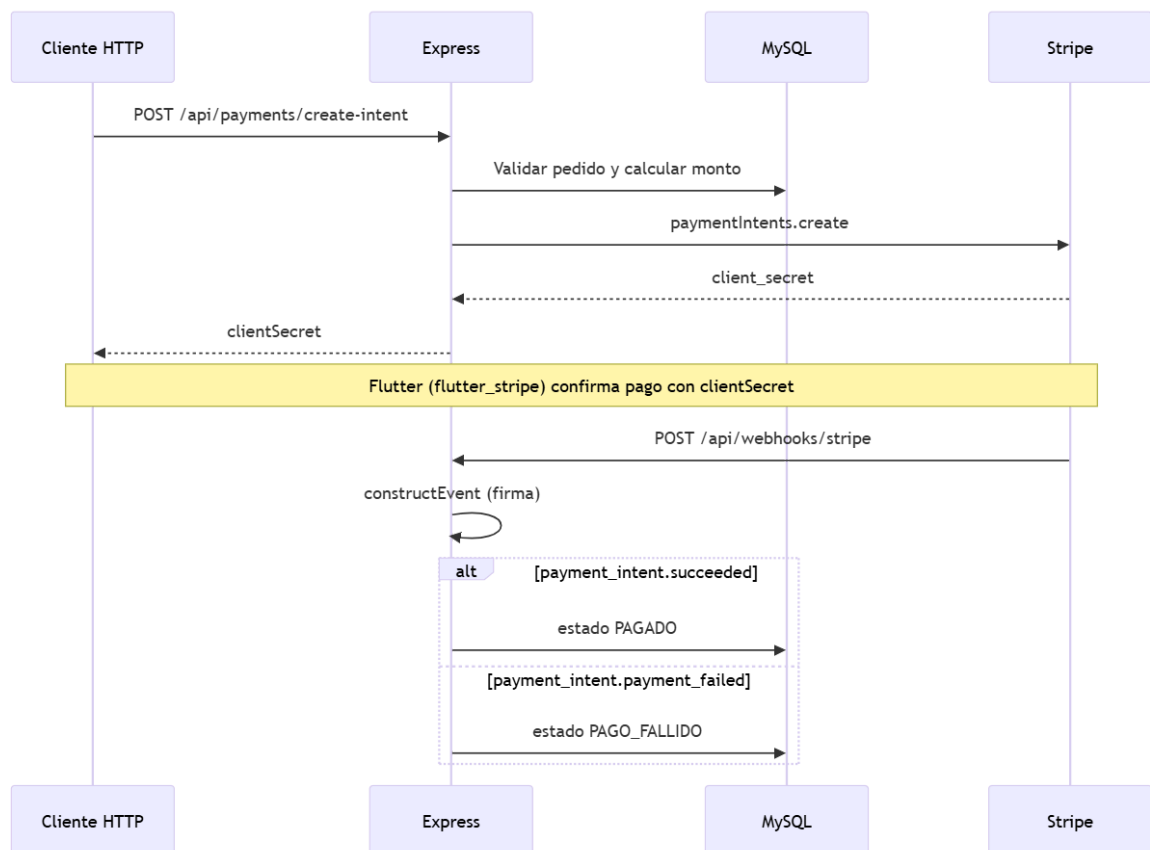


4.6 Flujos de negocio (resumen)

Compra y pago (flujo actual)



Pago con Stripe (backend implementado)



Envío automático: 60 segundos después de crear el pedido, `crearEnvioAutomatico` asigna método `MP001` (o el primero disponible), estado de envío `EN TRÁNSITO` y actualiza el pedido a `EN PREPARACIÓN`.

4.7 Integración con Stripe (resumen)

Aspecto	Implementación
SDK servidor	<code>stripe npm</code> , API 2024-06-20
SDK móvil	<code>flutter_stripe ^11.5.0</code>
Modo	Claves test (<code>pk_test_</code> en app, <code>sk_test_</code> en servidor)
Cobro	<code>PaymentIntent</code> + confirmación en dispositivo
Webhook	POST <code>/api/webhooks/stripe</code> (cuerpo raw, verificación de firma)
Fuente de verdad del pago	Webhook (no el mensaje de éxito del móvil)

4.8 Cliente Flutter (resumen)

Capa	Ubicación	Rol
Pantallas	<code>lib/screens/</code>	UI y flujos de usuario
Repositorios	<code>lib/services/*_repository.dart</code>	Llamadas HTTP por dominio
<code>ApiClient</code>	<code>lib/services/api_client.dart</code>	HTTP JSON + <code>postMultipart</code> + <code>buildPublicUrl</code>
Sesión local	<code>lib/services/client_session.dart</code>	<code>SharedPreferences</code> + <code>X-Client-ID</code> + <code>ClientProfile.avatarUrl</code>
Carrito local	<code>lib/services/cart_storage.dart</code>	Hasta crear el pedido

Rutas relevantes: `/` (home + búsqueda), `/cart`, `/checkout-payment`, `/login`, `/register`, `/orders` (perfil con pestañas Mis pedidos / Mis datos).

Pantalla	Archivo	Función
Login	login_screen.dart	Email + contraseña → <code>ClientsRepository.login</code> → Home
Registro	register_client_screen.dart	Datos + contraseña + confirmación + foto opcional (<code>image_picker</code>) → <code>POST /api/clientes</code> (JSON o multipart)
Perfil	my_orders_screen.dart	Pedidos, datos personales, avatar en cabecera, cerrar sesión

Bottom navigation dinámico (`app_bottom_nav.dart` + `bottom_nav_actions.dart`):

Sesión activa (' <code>ClientSession</code> ')	Tercer tab	Destino al pulsar
Sí	Perfil (<code>person_outline</code>)	<code>/orders</code>
No	Login (<code>login_rounded</code>)	<code>/login</code>

`MainLayout` refresca el estado de sesión al volver a una pantalla (`refreshSession`).

Cerrar sesión: en *Mis datos* → botón *Cerrar sesión* → diálogo de confirmación → `ClientSession.clear()` → `pushNamedAndRemoveUntil('/', '')`.

Repositorios adicionales: `PaymentsRepository` (`/api/payments/create-intent`), `ClientsRepository` (`register` con `avatarPath` opcional, `login`, `PUT/PATCH` perfil).

`ApiClient` expone `getJsonList`, `getJsonMap`, `postJson`, `putJson`, `patchJson` y `'postMultipart'` (registro con imagen). Para mostrar avatares usa `'buildPublicUrl(relativePath)'` → `{baseUrl}/public/{avatar}`. La URL base por defecto es `http://10.0.2.2:3000` (emulador Android).

Avatares en Flutter: `ClientProfile` guarda la ruta relativa (`avatars/...`); el getter `avatarUrl` construye la URL pública. En registro, si el usuario elige foto, `ClientsRepository.register` envía multipart; si no, JSON. En *Mis datos*, `CircleAvatar` carga la imagen con `NetworkImage(avatarUrl)` o muestra icono por defecto.

4.9 Ciclo de vida de una petición HTTP

Esta sección describe **qué ocurre en cada capa** cuando la app móvil llama al servidor, sin necesidad de abrir el código. El patrón se repite en casi todos los endpoints.

```
flowchart TD
    A[1. App Flutter<br/>Pantalla o Repositorio] --> B[2. ApiClient<br/>Construye URL y headers]
    B --> C[3. Red HTTP<br/>GET/POST/PUT]
    C --> D[4. Express server.js<br/>Middleware global]
    D --> E{5. ¿Ruta especial?}
    E -->|Webhook Stripe| F[Raw body + webhook.controller]
    E -->|API /api/*| G[express.json + routes/index]
    G --> H[6. Router del módulo<br/>ej. clienteRoutes]
    H --> I{7. ¿Requiere auth?}
    I -->|Sí| J[authMiddleware<br/>valida X-Client-ID]
    I -->|No| K[8. Controller]
    J --> K
    K --> L{9. ¿Lógica extra?}
    L -->|Pagos| M[Services<br/>payment / order]
    L -->|No| N[10. Modelo Sequelize]
    M --> N
    N --> O[(11. MySQL)]
    O --> N
    N --> K
    K --> P[12. Respuesta JSON<br/>success + data]
    P --> Q[13. Flutter parsea JSON<br/>Actualiza UI]
```

Paso a paso (lenguaje claro)

Paso	Quién	Qué hace
1	Pantalla Flutter	El usuario pulsa un botón; la pantalla llama a un <i>repositorio</i> (por ejemplo <code>OrdersRepository.createOrderFromCart</code>).
2	Repositorio	Arma el cuerpo JSON y los headers (si hay sesión, añade <code>X-Client-ID</code>).
3	ApiClient	Une la URL base (<code>http://10.0.2.2:3000</code>) con la ruta (<code>/api/pedidos</code>) y envía la petición HTTP.
4	Express	Recibe la petición. Aplica CORS, logging (<code>morgan</code>) y, salvo el webhook, convierte el body a JSON.

Paso	Quién	Qué hace
5	Router	<code>routes/index.js</code> delega en el router del módulo (<code>/clientes</code> , <code>/pedidos</code> , <code>/productos</code> , etc.).
6	Middleware de auth (si aplica)	Comprueba que exista el header <code>X-Client-ID</code> , busca el cliente en MySQL y adjunta <code>req.cliente</code> al request.
7	Controller	Valida campos obligatorios, reglas de negocio (DNI, email, stock, etc.) y decide qué hacer.
8	Service (opcional)	En pagos, calcula el monto en centavos desde la BD o crea el Payment Intent en Stripe.
9	Modelo Sequelize	Traduce operaciones a SQL (<code>findAll</code> , <code>create</code> , <code>update</code>) sobre las tablas.
10	Respuesta	El controller responde JSON, normalmente <code>{ "success": true, "data": ... }</code> .
11	Flutter	Si <code>success</code> es true, actualiza la pantalla; si no, muestra error (SnackBar, alerta o texto en formulario).

Punto de entrada del servidor (`server.js`)

El orden de los middlewares es importante: el webhook de Stripe debe leer el body **sin parsear** para verificar la firma.

```
// server-paloma/server.js (fragmento)
app.use(cors());
app.use(morgan('dev'));

// Webhook ANTES de express.json()
app.post('/api/webhooks/stripe', express.raw({ type:
'application/json' }), handleStripeWebhook);

app.use(express.json());
// Avatares y otros estáticos bajo /public
app.use('/public', express.static(path.join(__dirname, 'public')));
app.use('/api', routes);

app.use(notFoundHandler); // 404 → "Ruta no encontrada"
app.use(errorHandler);    // Errores Sequelize y 500
```

Ejemplo de delegación en rutas (`clienteRoutes.js`)

```
router.post('/login', loginCliente); //
```

```
Público: email + password
router.post('/', handleAvatarUpload, createCliente); // Público:
registro (+ password, avatar opcional)
router.get('/mi-perfil', authenticate, getMiPerfil);
router.put('/mi-perfil', authenticate, updateMiPerfil);
router.patch('/mi-perfil', authenticate, updateMiPerfil);
```

`handleAvatarUpload` envuelve `uploadAvatarOptional` (Multer) y devuelve 400 si falla la subida. El registro **sin** archivo sigue funcionando: Multer no exige el campo `avatar`.

La ruta completa que ve el móvil es: **PUT** `http://host:3000/api/clientes/mi-perfil` (prefijo `/api` + router `/clientes` + path `/mi-perfil`).

4.10 Flujos detallados con código

4.10.1 Búsqueda de productos (Home)

Fase	Componente	Acción
UI	<code>home_screen.dart</code>	Usuario escribe en el buscador; debounce 450 ms.
Repositorio	<code>ProductsRepository.getProductos(q: texto)</code>	GET <code>/api/productos?q=...</code>
Controller	<code>productoController.getAllProductos</code>	Filtra por nombre y descripción con LIKE.
Modelo	<code>Producto.findAll</code>	Consulta tabla <code>productos</code> .
Respuesta	JSON lista en <code>data</code>	Flutter pinta secciones de productos.

Petición desde Flutter (concepto):

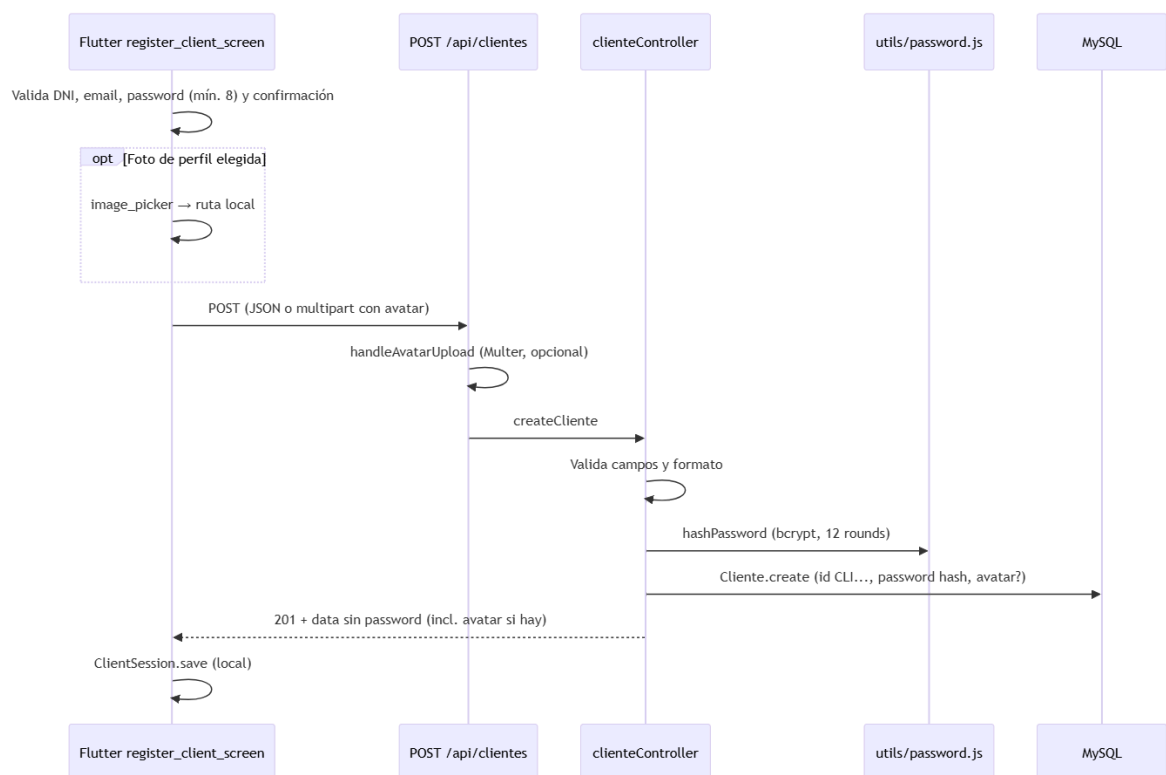
```
// GET http://10.0.2.2:3000/api/productos?q=avena
final products = await ProductsRepository.getProductos(q: 'avena');
```

Consulta en servidor (concepto):

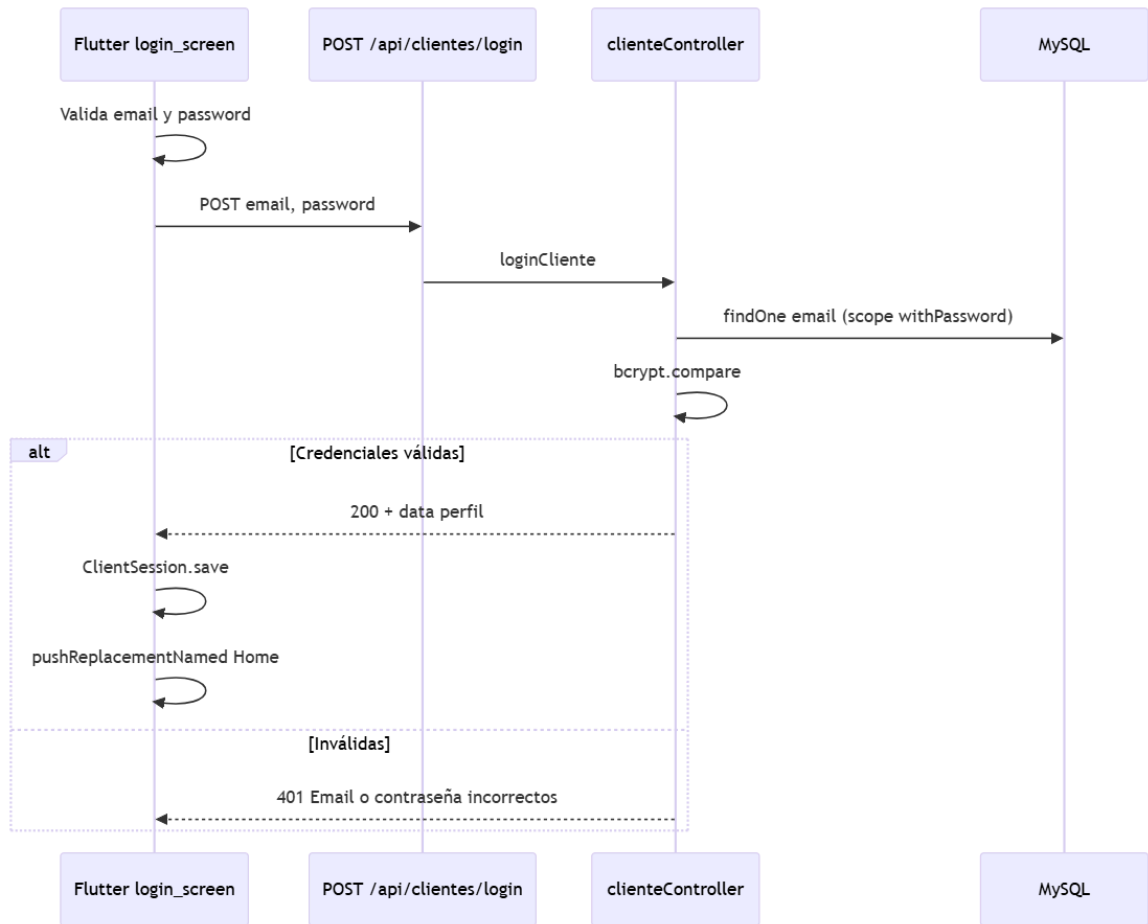
```
// productoController.js - si hay ?q=, aplica Op.like en nombre y
descripcion
const productos = await Producto.findAll({ where, include:
['categoria'], order: [['nombre', 'ASC']] });
res.json({ success: true, count: productos.length, data: productos
});
```

4.10.2 Registro, login, sesión y cierre (Mis datos)

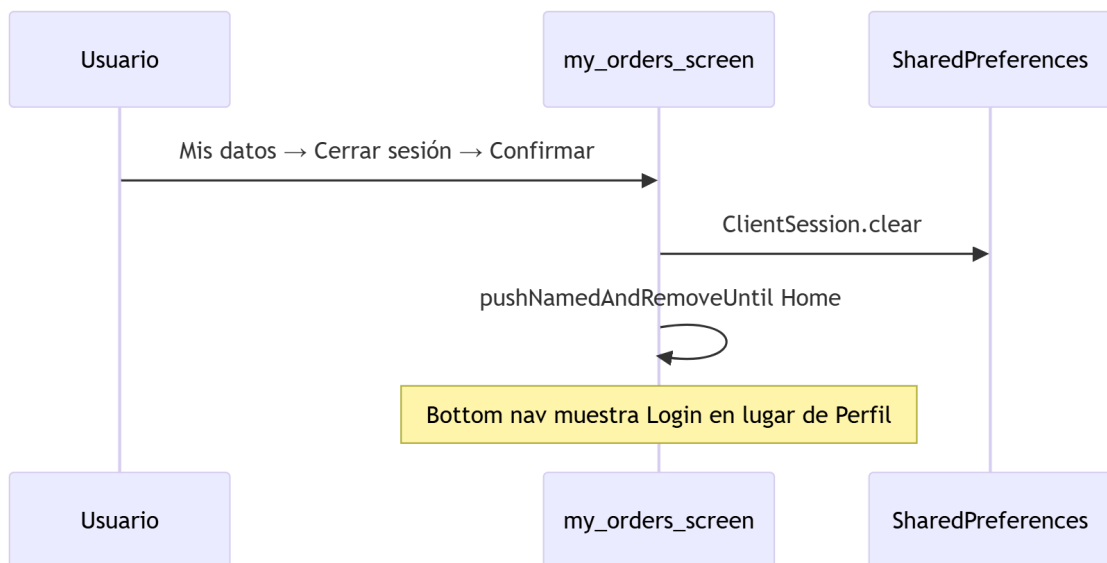
Registro (primera vez) — sin autenticación previa:



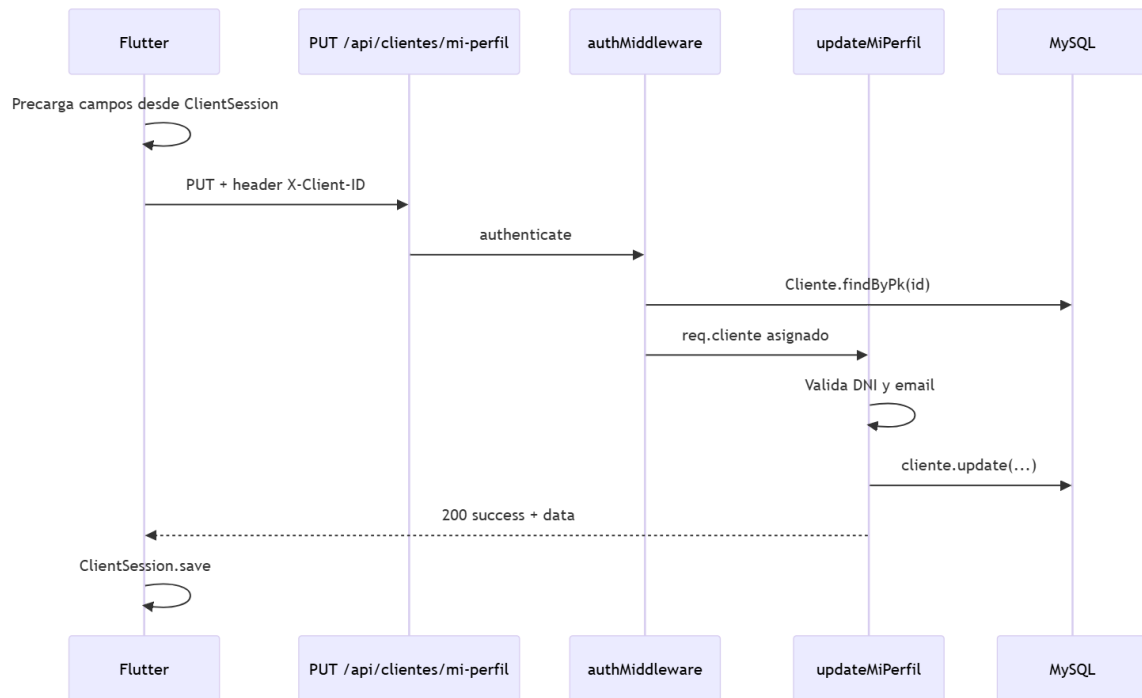
Inicio de sesión — sin token; misma respuesta que registro:



Cierre de sesión — solo cliente (no hay endpoint en servidor):



Actualización (Mis datos → Editar) — requiere sesión:



Código Flutter — registro, login y actualización:

```
// clients_repository.dart - registro sin avatar (JSON)
await _client.postJson('/api/clientes', body: {
  'nombre': nombre, 'dni': dni, 'telefono': telefono,
  'email': email, 'direccion': direccion, 'password': password,
});

// clients_repository.dart - registro con avatar (multipart)
await _client.postMultipart('/api/clientes',
  fields: {
    'nombre': nombre, 'dni': dni, 'telefono': telefono,
    'email': email, 'direccion': direccion, 'password': password,
  },
  filePath: avatarPath, // campo multipart "avatar"
);

// client_session.dart - URL pública del avatar
String? get avatarUrl => ApiClient.buildPublicUrl(avatar);
// Ejemplo:
http://10.0.2.2:3000/public/avatars/avatar-1710000000-abc.jpg

// clients_repository.dart - login
await _client.postJson('/api/clientes/login', body: {
  'email': email.trim(), 'password': password,
});

// clients_repository.dart - actualización perfil
```

```

await _client.putJson('/api/clientes/mi-perfil',
  body: { 'nombre': nombre, 'dni': dni, 'telefono': telefono,
'email': email, 'direccion': direccion },
  headers: { 'X-Client-ID': local.idCliente },
);

```

Middleware de autenticación (servidor):

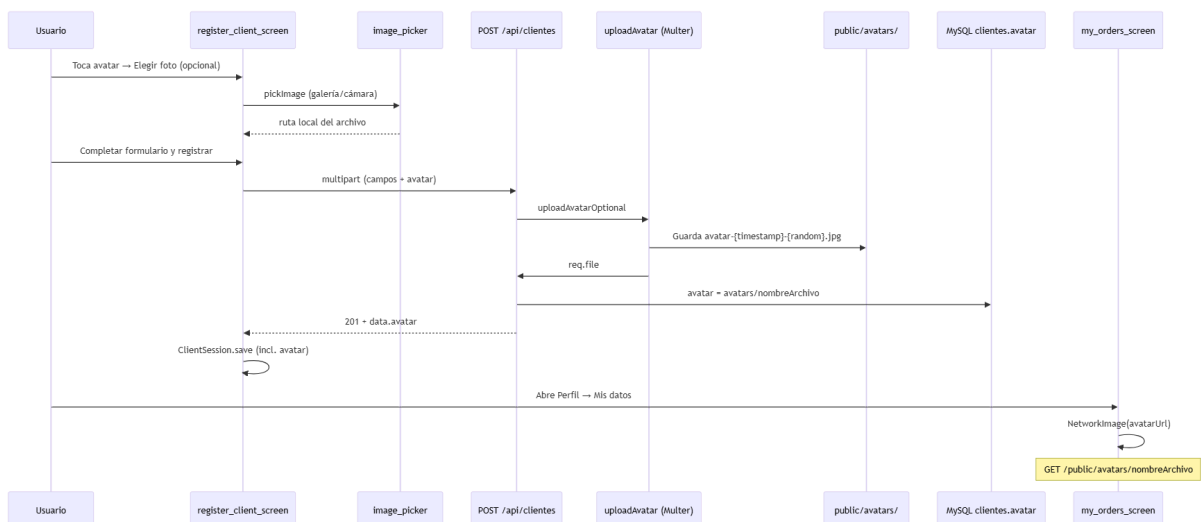
```

// authMiddleware.js - extracto
const clientId = req.headers['x-client-id'] ||
req.headers['X-Client-ID'];
const cliente = await Cliente.findByPk(clientId.trim());
if (!cliente) return res.status(401).json({ success: false, message:
'Cliente no encontrado...' });
req.cliente = cliente;
req.clientId = cliente.idCliente;
next();

```

4.10.3 Avatares de perfil

Flujo completo desde la selección de imagen en el móvil hasta su visualización en *Mis datos*.



Capa	Archivo / componente	Responsabilidad
Subida	<code>middleware/uploadAvatar.js</code>	Multer diskStorage, límite 2 MB, MIME/ext permitidos
Ruta BD	<code>utils/avatar.js</code> → <code>toAvatarDbPath</code>	Convierte nombre de archivo a <code>avatars/...</code>
Limpieza	<code>utils/avatar.js</code> → <code>cleanupUploadedFile</code>	Borra archivo si falla validación o create
Estáticos	<code>server.js</code> → <code>express.static('public')</code>	Sirve imágenes en <code>/public/avatars/*</code>
Registro UI	<code>register_client_screen.dart</code>	CircleAvatar + botón elegir/quitar foto
HTTP móvil	<code>api_client.dart</code> → <code>postMultipart</code>	Envía campos + archivo en un solo POST
Sesión	<code>client_session.dart</code>	Persiste <code>avatar</code> ; expone <code>avatarUrl</code>
Visualización	<code>my_orders_screen.dart</code>	Cabecera con foto o icono por defecto

Limitaciones actuales (demo académica):

- El avatar solo se sube en **registro**; `PUT /api/clientes/mi-perfil` no acepta multipart ni cambia la imagen.
- Almacenamiento **local en disco** del servidor (no S3/CDN); las imágenes subidas están en `.gitignore` (`public/avatars/*`).
- Clientes registrados antes de la migración pueden tener `avatar` null.
- En dispositivo físico, la URL del avatar usa la misma `ApiClient.defaultBaseUrl` que el resto de la API (debe apuntar al host accesible desde el móvil).

4.10.4 Crear pedido desde el carrito

Paso	Detalle
1	<code>CartScreen</code> verifica que exista <code>ClientSession</code> (si no, va a registro o login).

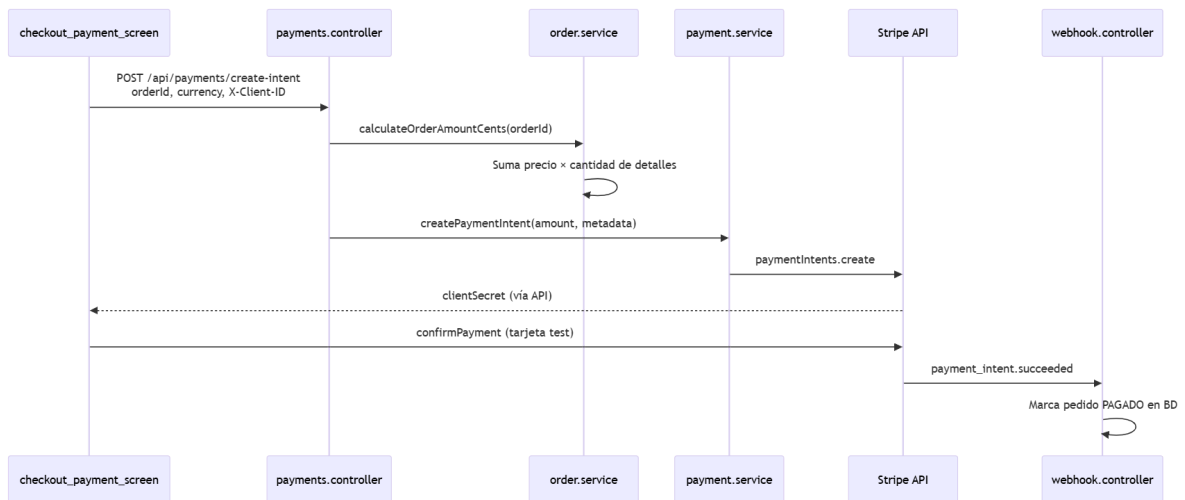
Paso	Detalle
2	<code>OrdersRepository.createOrderFromCart</code> → POST <code>/api/pedidos</code> con <code>idCliente</code> y array detalles.
3	<code>pedidoController.createPedido</code> abre transacción Sequelize.
4	Valida stock de cada producto; si falta stock, rollback y error 400.
5	Crea Pedido estado PENDIENTE, líneas en <code>DetallePedido</code> , descuenta stock.
6	Commit; programa envío automático a los 60 s (<code>EN PREPARACIÓN</code> + envío <code>EN TRÁNSITO</code>).
7	Devuelve <code>idPedido</code> ; Flutter navega a pantalla de pago Stripe.

Body de ejemplo:

```
{
  "idCliente": "CLI1234567890123",
  "detalles": [
    { "idProducto": "PROD001", "cantidad": 2 }
  ]
}
```

4.10.5 Pago con Stripe (móvil + servidor)

Principio: el móvil **no** envía el importe final a Stripe directamente. El servidor recalcula el total desde los detalles del pedido en MySQL.



Servidor — recalcular monto (no confiar en el cliente):

```
// order.service.js - idea
const { amountCents } = await calculateOrderAmountCents({ orderId,
userId });
// Suma (precio x cantidad) de cada DetallePedido con Producto
incluido
```

Servidor — crear intención de pago:

```
// payment.service.js - idea
const intent = await stripe.paymentIntents.create({
  amount: amountCents,
  currency: 'usd',
  metadata: { userId, orderId, appName: 'server-paloma' },
  automatic_payment_methods: { enabled: true, allow_redirects:
'never' },
});
return { clientSecret: intent.client_secret, id: intent.id };
```

Móvil — confirmar tarjeta:

```
// checkout_payment_screen.dart - idea
await Stripe.instance.confirmPayment(
  paymentIntentClientSecret: intent.clientSecret,
  data: PaymentMethodParams.card(...),
```

```
);
```

Webhook — actualizar pedido (fuente de verdad):

```
// webhook.controller.js - payment_intent.succeeded
const orderId = intent.metadata.orderId;
await markOrderPaid({ orderId }); // estado → PAGADO
```

4.10.6 Tarea programada (cron) de estados

Cada día a la **1:00 AM** (zona Europe/Madrid por defecto):

Entidad	De	A
Pedido	EN PREPARACIÓN	FINALIZADO
Envío	EN TRÁNSITO	ENTREGADO

Implementado en `services/dailyStatusCron.service.js`, arrancado desde `server.js` con `node-cron`.

Variables de entorno:

Variable	Valor típico	Efecto
<code>ENABLE_DAILY_STATUS_CRON</code>	<code>true</code>	Activa la tarea; <code>false</code> la desactiva
<code>CRON_TIMEZONE</code>	Europe/Madrid	Zona horaria del disparo a la 1:00

4.11 Configuración de base de datos

Variables de entorno (servidor)

Variable	Significado	Ejemplo
DB_HOST	IP o hostname del servidor MySQL	localhost
DB_PORT	Puerto MySQL	3306
DB_NAME	Nombre de la base de datos	paloma_db
DB_USER	Usuario con permisos de lectura/escritura	root
DB_PASSWORD	Contraseña del usuario	(vacío en local)

Conexión Sequelize (config/database.js)

```
const sequelize = new Sequelize(DB_NAME, DB_USER, DB_PASSWORD, {
  host: DB_HOST,
  port: DB_PORT,
  dialect: 'mysql',
  logging: NODE_ENV === 'development' ? console.log : false,
  pool: { max: 5, min: 0, acquire: 30000, idle: 10000 },
});
```

Parámetro	Para qué sirve
dialect: 'mysql'	Indica que el motor es MySQL (usa driver mysql2).
pool	Reutiliza conexiones; evita abrir/cerrar una por cada request.
logging	En desarrollo imprime el SQL en consola; en producción conviene desactivarlo.

Preparación inicial de la BD

```
CREATE DATABASE paloma_db CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;
```

```
cd server-paloma
npm run seed # categorías y productos de ejemplo
npm run seed:metodos-pago # catálogo de métodos de pago local
```

En `NODE_ENV=development`, al iniciar el servidor Sequelize puede sincronizar modelos (`sync({ alter: false })`) sin alterar tablas existentes.

Tablas principales y responsabilidad

Tabla	Quién escribe	Cuándo
clientes	POST <code>/api/clientes</code> , POST <code>/login</code> , PUT <code>mi-perfil</code>	Registro (hash password, avatar opcional), login, edición perfil (sin avatar)
productos	Scripts seed / administración externa	Catálogo
Pedido, DetallePedido	POST <code>/api/pedidos</code>	Checkout
Pedido.estado	Webhook Stripe, cron, envío automático	Pago / mantenimiento
envio	Job automático 60 s tras pedido, cron	Logística simulada
resenia	POST <code>/api/resenias</code>	Tras compra (con sesión)

4.12 Configuración de Stripe (servidor y móvil)

Stripe se usa solo como **procesador de pago**. No se crean clientes, productos ni precios en Stripe: el catálogo y los importes viven en MySQL.

Claves y dónde van

Clave	Prefijo	Dónde va	Quién la ve
Publishable	<code>pk_test_ /</code> <code>pk_live_</code>	App Flutter <code>stripe_config.dart</code>	Puede ir en el cliente
Secret	<code>sk_test_ /</code> <code>sk_live_</code>	Servidor <code>.env</code> → <code>STRIPE_SECRET_KEY</code>	Solo servidor

Clave	Prefijo	Dónde va	Quién la ve
Webhook signing	whsec_	Servidor .env → STRIPE_WEBHOOK_SECRET	Solo servidor

Nunca poner `sk_test_` en la app móvil.

Servidor (`server-paloma/.env`)

```
STRIPE_SECRET_KEY=sk_test ...
STRIPE_WEBHOOK_SECRET=whsec...
```

Variable	Uso
STRIPE_SECRET_KEY	Crear Payment Intents, hablar con API Stripe
STRIPE_WEBHOOK_SECRET	Verificar que el POST del webhook viene realmente de Stripe

Móvil (`SaborSano-web`)

Archivo: `lib/config/stripe_config.dart`

```
static const String publishableKey = 'pk_test...'; // o
--dart-define=STRIPE_PUBLISHABLE_KEY=...
```

Inicialización en `main.dart`:

```
WidgetsFlutterBinding.ensureInitialized();
Stripe.publishableKey = StripeConfig.publishableKey;
await Stripe.instance.applySettings();
runApp(const SaborSanoApp());
```

Android (requisitos `flutter_stripe`)

Requisito	Motivo
MainActivity extiende FlutterFragmentActivity	UI nativa de Stripe
Tema Theme.MaterialComponents en styles.xml	Componentes de tarjeta
Dependencias appcompat y material en Gradle	Soporte de temas

Webhook en desarrollo local

Stripe Dashboard **no** puede llamar a localhost. Opciones:

1. **Stripe CLI:** `stripe listen --forward-to http://localhost:3000/api/webhooks/stripe` → copiar `whsec_` al `.env`.
2. **Túnel (ngrok, etc.):** URL pública → mismo path `/api/webhooks/stripe`.

Sin webhook, el pago puede **succeed** en Stripe pero el pedido en MySQL puede seguir PENDIENTE.

Tarjetas de prueba

Campo	Valor
Número	4242 4242 4242 4242
Fecha	Cualquier fecha futura (ej. 12/34)
CVC	Cualquier 3 dígitos

4.13 Buenas prácticas aplicadas

Práctica	Dónde se aplica
Separación en capas	Rutas → controllers → services → models

Práctica	Dónde se aplica
No confiar en el monto del cliente	<code>calculateOrderAmountCents</code> antes del <code>Payment Intent</code>
Webhook como verdad del pago	Estado <code>PAGADO/PAGO_FALLIDO</code> solo tras evento Stripe
Transacciones en pedidos	Rollback si falla stock o detalle
Validación en servidor	DNI español, email, campos requeridos, password mín. 8 caracteres
Hash de contraseñas	<code>bcryptjs</code> (12 rounds); nunca se devuelve el hash en JSON
Autenticación por credenciales	Login email + password; mensaje genérico en error 401
Autenticación por recurso	<code>X-Client-ID</code> + existencia en BD (sin JWT)
Sesión local en móvil	<code>ClientSession / SharedPreferences</code> ; logout solo en cliente
Idempotencia en pagos	Cabecera <code>Idempotency-Key</code> o <code>order: {orderId}</code>
Errores JSON uniformes	<code>{ success, message }</code> + middleware central
Secretos en <code>.env</code>	No commitear claves Stripe ni contraseña BD
Carrito local hasta checkout	Menos carga al servidor; pedido explícito al pagar
Debounce en búsqueda	Menos peticiones mientras el usuario escribe
PCI simplificado	Datos de tarjeta van a Stripe vía SDK, no al backend propio
Validación de imágenes	Multer: MIME/ext permitidos, máx. 2 MB; limpieza si falla el registro
Avatares en disco local	Demo académica; <code>.gitignore</code> excluye <code>public/avatars/*</code> ; producción usaría almacenamiento externo
Avatar como ruta relativa	BD guarda <code>avatars/...</code> ; URL completa la construye el cliente con <code>/public</code>

4.14 Glosario para revisión

Término	Significado en este proyecto
API REST	Interfaz HTTP con rutas y verbos (GET, POST, PUT) que devuelve JSON
Controller	Función que recibe la petición HTTP, valida y responde
Repository (Flutter)	Clase que encapsula llamadas HTTP de un dominio (pedidos, clientes)
Sequelize / Modelo	Capa que traduce objetos JavaScript a filas de tablas SQL
Payment Intent	Objeto Stripe que representa un cobro pendiente o completado
clientSecret	Cadena que el móvil usa para confirmar el pago con Stripe
Webhook	Llamada que Stripe hace al servidor cuando cambia el estado del pago
X-Client-ID	Identificador de sesión del cliente (no es JWT); se envía tras login/registro
ClientSession	Perfil guardado en <code>SharedPreferences</code> ; fuente de <code>X-Client-ID</code> en la app
bcrypt	Algoritmo de hash usado para <code>clientes.password</code> en el servidor
Multer	Middleware Express para <code>multipart/form-data</code> ; sube el avatar al registro
avatarUrl	Getter en <code>ClientProfile</code> que une <code>ApiClient.baseUrl + /public/ + ruta en BD</code>
Middleware	Código que se ejecuta antes del controller (auth, JSON, Multer, errores)

5. Configuración e instalación

Detalle conceptual de BD y Stripe: secciones [4.11](#) y [4.12](#). Flujos internos: [4.9](#) y [4.10](#).

5.1 Variables de entorno (backend)

Archivo de referencia: `server-paloma/env.example`

Variable	Descripción
PORT	Puerto HTTP (default 3000)
NODE_ENV	development activa logs SQL y sequelize.sync
DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD	Conexión MySQL (véase 4.11)
STRIPE_SECRET_KEY	Secret key de Stripe (test o live)
STRIPE_WEBHOOK_SECRET	Secreto de firma del endpoint webhook
CRON_TIMEZONE	Zona del cron diario (default Europe/Madrid)
ENABLE_DAILY_STATUS_CRON	true / false para activar transiciones automáticas de pedido/envío

5.2 Puesta en marcha del backend

```
cd server-paloma
npm install
cp env.example .env
# Editar .env con credenciales MySQL y Stripe
# Crear base de datos: CREATE DATABASE paloma_db;
npm run seed
npm run seed:metodos-pago
npm run migrate:cliente-password # si la BD ya existía sin columna password
npm run migrate:cliente-avatar # si la BD ya existía sin columna avatar
npm run dev
```

Servidor disponible en `http://localhost:3000` y API en `http://localhost:3000/api`.

5.3 Puesta en marcha del frontend

```
cd SaborSano-web
flutter pub get
flutter run
```

En **dispositivo físico**, ajustar `ApiClient.defaultBaseUrl` a la IP LAN de la máquina que ejecuta el backend (por ejemplo `http://192.168.1.10:3000`).

Stripe en el móvil: configurar `lib/config/stripe_config.dart` con la clave publicable `pk_test_...` (nunca la secreta `sk_test_`). Tras cambiar la clave, ejecutar `flutter run` de nuevo. Requisitos Android: ver [4.12](#).

5.4 Webhook Stripe en desarrollo

Usar Stripe CLI o el Dashboard para reenviar eventos a:

```
POST http://localhost:3000/api/webhooks/stripe
```

El cuerpo debe llegar sin parsear JSON (el servidor registra `express.raw` solo en esa ruta).

Documento generado a partir del código fuente de `server-paloma` y `SaborSano-web`. No incluye información no presente en el repositorio.